

Análise Multidimensional da Qualidade do Saneamento em São Paulo: Uma Abordagem com Aprendizado de Máquina (2013–2024)

✓ 1. INTRODUÇÃO DO BANCO DE DADOS E DO PROBLEMA A SER ANALISADO

Este estudo utiliza o abrangente conjunto de dados do Sistema Nacional de Informações sobre Saneamento (SNIS), que detalha a operação dos serviços de água e esgoto no Brasil através de mais de 130 variáveis. O nosso objetivo é superar as análises tradicionais, que frequentemente ocultam as diferentes realidades municipais, para identificar e caracterizar os perfis de saneamento existentes no estado de São Paulo.

Para alcançar isso, aplicaremos uma metodologia de aprendizado de máquina não supervisionado. O processo consiste em um rigoroso pré-processamento dos dados, agregando um período de cinco anos (2013-2022) e criando métricas normalizadas per capita para permitir uma comparação justa entre as cidades. Em seguida, utilizaremos um conjunto de algoritmos de clusterização — K-Means, DBSCAN e Agrupamento Hierárquico — para agrupar os municípios e revelar os diferentes perfis de gestão com base em suas características operacionais, financeiras e de qualidade de serviço.

Nome	RA
Arthur Pereira Gon	811464

✓ 1.1 INSTRUÇÕES PARA A EXECUÇÃO DO COLAB

<https://basedosdados.org/dataset/2a543ad8-3cdb-4047-9498-efe7fb8ed697?table=df7cf198-4889-4baf-bb77-4e0e28eb90ca>

o link acima é onde está nossa base de dados, siga as instruções a seguir para baixar os arquivos:

- Vá até Acesso aos dados
- Clique na aba de "Download"
- Baixe o arquivo na linha de "id_municipio"
- Baixe a Tabela no botão "Download da Tabela"
- Com os dois arquivos baixados basta fazer upload deles nesse Colab

- Certifique-se de que os nomes dos arquivos são:
 - br_mdr_snis_municipio_agua_esgoto.csv.gz
 - br_bd_diretorios_brasil_municipio.csv.gz

2. PRÉ PROCESSAMENTO DO DADOS

✓ 2.1 - Carregamento dos dados

```
# --- Importação de Bibliotecas ---
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
from sklearn.metrics import silhouette_score, calinski_harabasz_score, davies_bouldin_score
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage

print("Bibliotecas importadas com sucesso.")

# --- Carregamento dos Dados ---
# Garanta que os arquivos foram carregados no ambiente do Colab
nome_arquivo_bruto = 'br_mdr_snis_municipio_agua_esgoto.csv.gz'
nome_arquivo_nomes = 'br_bd_diretorios_brasil_municipio.csv.gz'

try:
    print(f"Carregando e descompactando os arquivos...")
    dados_brutos = pd.read_csv(nome_arquivo_bruto, compression='gzip', sep=',')
    dados_nomes_bruto = pd.read_csv(nome_arquivo_nomes, compression='gzip', sep=',')
    print("Dados carregados com sucesso!")
except Exception as e:
    print(f"ERRO ao carregar arquivos: {e}")
    raise
```

⇒ Bibliotecas importadas com sucesso.
Carregando e descompactando os arquivos...
Dados carregados com sucesso!

✓ 2.2 - Definições e limpeza de variáveis

```
# Verifica se a variável 'dados_brutos' da Célula 1 foi criada
if 'dados_brutos' in locals():
    # --- Parâmetros da Análise ---
    print("Definindo parâmetros da análise...")
    UF_ALVO = "SP"
    ANO_INICIAL = 2013
    ANO_FINAL = 2022
```

```

# --- Curadoria de Variáveis ---
# A lista com as 22 colunas-chave que usaremos como matéria-prima.
colunas_para_manter = [
    "id_municipio", "sigla_uf", "ano",
    "populacao_urbana", "populacao_urbana_atendida_agua_ibge", "populacao_urbana_aten
    "extensao_rede_agua", "extensao_rede_esgoto", "quantidade_economia_ativa_agua",
    "quantidade_economia_ativa_esgoto", "volume_agua_faturado", "volume_esgoto_fatura
    "indice_hidrometracao", "indice_macromedicao_volume_disponibilizado",
    "indice_perda_faturamento", "indice_tratamento_esgoto",
    "indice_consumo_agua_per_capita", "indice_fluoretacao_agua",
    "receita_operacional", "credito_areceber", "despesa_exploracao", "investimento_tc
]

# Limpeza do dataframe de nomes (pode ser feito aqui ou na Célula 1)
dados_nomes = dados_nomes_bruto[['id_municipio', 'nome']].copy().drop_duplicates(subs

print(f"Parâmetros definidos: Foco em '{UF_ALVO}', período de {ANO_INICIAL} a {ANO_FI

# --- Filtragem e Seleção ---
print("Iniciando filtragem e seleção de variáveis...")

# Filtra o dataframe bruto com base nos parâmetros definidos
df_filtrado = dados_brutos[(dados_brutos['sigla_uf'] == UF_ALVO) & (dados_brutos['anc

# Seleciona apenas as 22 colunas da nossa lista de matéria-prima
df_selecionado = df_filtrado[[col for col in colunas_para_manter if col in df_filtrad

print("\n-----")
print("Filtragem e seleção inicial concluídas!")
print(f"O dataframe 'df_selecionado' está pronto para a próxima etapa.")
print(f"Dimensões atuais: {df_selecionado.shape[0]} linhas e {df_selecionado.shape[1]
print("-----")

else:
    print("ERRO: Execute a Célula 1 de carregamento primeiro.")

```

➡ Definindo parâmetros da análise...

Parâmetros definidos: Foco em 'SP', período de 2013 a 2022.

Iniciando filtragem e seleção de variáveis...

```

-----
Filtragem e seleção inicial concluídas!
O dataframe 'df_selecionado' está pronto para a próxima etapa.
Dimensões atuais: 6215 linhas e 22 colunas.
-----

```

Este bloco de código é onde definimos o "coração" da nossa análise. Após uma avaliação inicial do dataset bruto do SNIS, que contém mais de 130 colunas, foi realizado um rigoroso processo de curadoria e seleção de atributos com os seguintes objetivos:

Foco e Relevância: Descartamos dezenas de variáveis que eram desdobramentos financeiros, administrativos ou geográficos muito granulares. O foco foi manter apenas as métricas que

refletem diretamente a qualidade, cobertura e eficiência dos serviços de saneamento urbano.

Eliminação de Redundância: Removemos variáveis que mediam o mesmo conceito. Por exemplo, em vez de manter cinco tipos de "volume de água" (produzido, consumido, tratado, etc.), mantivemos o volume_agua_faturado como a principal métrica de negócio. Da mesma forma, selecionamos receita_operacional e investimento_total_municipio como os indicadores financeiros e de investimento mais representativos.

Engenharia de Features: A lista foi estrategicamente montada para conter dois tipos de variáveis:

Índices de Performance Diretos: Variáveis que já são taxas ou porcentagens de alta qualidade, como indice_tratamento_esgoto e indice_perda_faturamento.

Métricas Absolutas para Normalização: Variáveis como extensao_rede_agua e receita_operacional, que foram mantidas com o propósito específico de serem divididas pela populacao_urbana na etapa seguinte, criando assim nossos próprios indicadores per capita e permitindo uma comparação justa entre municípios de diferentes portes.

Ao final, este processo de seleção nos permite trabalhar com um conjunto de dados mais enxuto, coeso e significativo, o que é fundamental para gerar resultados de clusterização mais claros e interpretáveis.

✓ 2.3 - Agregação e engenharia de features

```
if 'df_selecionado' in locals():
    print("Agregando dados e criando novas features...")

    # Agrega o dataframe já enxuto
    df_agregado = df_selecionado.groupby(['id_municipio', 'sigla_uf'], as_index=False).me

    # Cria as features de cobertura e per capita
    df_agregado['ind_cobertura_agua'] = df_agregado['populacao_urbana_atendida_agua_ibge']
    df_agregado['ind_cobertura_esgoto'] = df_agregado['populacao_urbana_atendida_esgoto']

    vars_para_normalizar = [
        "extensao_rede_agua", "extensao_rede_esgoto", "quantidade_economia_ativa_agua",
        "quantidade_economia_ativa_esgoto", "volume_agua_faturado", "volume_esgoto_fatura
        "receita_operacional", "credito_areceber", "despesa_exploracao", "investimento_tc
    ]
    for col in vars_para_normalizar:
        if col in df_agregado.columns:
            df_agregado[f'{col}_per_capita'] = df_agregado[col] / (df_agregado['populacac

    df_com_todas_features = pd.merge(df_agregado, dados_nomes, on='id_municipio', how='le

    print("Engenharia de features concluída.")
    display(df_com_todas_features.head())
```

```
else:
    print("ERRO: Execute a Célula 3 primeiro.")
```

➡ Agregando dados e criando novas features...
Engenharia de features concluída.

	id_municipio	sigla_uf	ano	populacao_urbana	populacao_urbana_atendida_agua_i
0	3500105	SP	2017.5	33147.222222	3502
1	3500204	SP	2017.5	3233.666667	367
2	3500303	SP	2019.6	32729.000000	3543
3	3500402	SP	2017.5	7233.666667	802
4	3500501	SP	2017.5	18342.111111	1845

5 rows × 35 columns

Nesta etapa, transformamos os dados anuais de cada município em um perfil único e comparável. Para isso, calculamos a média dos indicadores ao longo do período de 2013 a 2022, ignorando anos com dados faltantes (N/A) para criar um retrato estável de cada cidade. Em seguida, realizamos a etapa mais importante da normalização: a criação de métricas per capita. Ao dividir valores absolutos (como receita e extensão de rede) pela população, garantimos uma comparação justa da eficiência e do nível de serviço entre municípios de portes diferentes, o que é essencial para uma clusterização significativa.

✓ 2.4 - Análise de multicolinearidade

```
if 'df_com_todas_features' in locals():
    print("Iniciando a etapa final de seleção e remoção de multicolinearidade...")

# --- 1. Definir o nosso conjunto de 18 features de interesse ("Gabarito") ---
colunas_finais_candidatas = [
    'ind_cobertura_agua', 'ind_cobertura_esgoto', 'indice_consumo_agua_per_capita',
    'extensao_rede_agua_per_capita', 'extensao_rede_esgoto_per_capita',
    'quantidade_economia_ativa_agua_per_capita', 'quantidade_economia_ativa_esgoto_pe
    'volume_agua_faturado_per_capita', 'volume_esgoto_faturado_per_capita',
    'credito_areceber_per_capita', 'despesa_exploracao_per_capita',
    'investimento_total_municipio_per_capita', 'indice_hidrometracao',
    'indice_macromedicao_volume_disponibilizado', 'indice_perda_faturamento',
    'indice_tratamento_esgoto', 'indice_fluoretacao_agua',
    'receita_operacional_per_capita' # Adicionando as colunas que seriam removidas pa
]

# Isola o dataframe para conter apenas as features que vamos analisar
df_features_analise = df_com_todas_features[colunas_finais_candidatas].dropna()
```

```

# --- 2. Calcular a matriz de correlação e identificar pares problemáticos ---
limite_cor = 0.7
corr_matrix = df_features_analise.corr()

# Transforma a matriz em uma tabela de pares para fácil análise
pares_correlacionados = corr_matrix.abs().stack().reset_index()
pares_correlacionados.columns = ['V1', 'V2', 'Correlacao']
pares_correlacionados = pares_correlacionados[pares_correlacionados['V1'] != pares_correlacionados['V2']]
pares_correlacionados['sorted_vars'] = pares_correlacionados.apply(lambda row: tuple(sorted([row['V1'], row['V2']])), axis=1)
pares_correlacionados = pares_correlacionados.drop_duplicates(subset=['sorted_vars'])
pares_altamente_correlacionados = pares_correlacionados[pares_correlacionados['Correlacao'] > limite_cor]

print(f"\n--- Pares de variáveis com correlação absoluta > {limite_cor} ---")
print(pares_altamente_correlacionados.to_string())

# --- 3. Lógica de Exclusão Inteligente ---
cor_media_abs = corr_matrix.abs().mean()
colunas_para_remover = set()

for _, row in pares_altamente_correlacionados.iterrows():
    var1, var2 = row['V1'], row['V2']
    if cor_media_abs[var1] > cor_media_abs[var2]:
        colunas_para_remover.add(var1)
    else:
        colunas_para_remover.add(var2)

colunas_para_remover = list(colunas_para_remover)
print(f"\n--- Colunas selecionadas para remoção (mais redundantes): ---")
print(colunas_para_remover)

# --- 4. Criar o dataframe final sem as colunas redundantes ---
# Começamos com o dataframe com todas as colunas...
df_perfil_final = df_com_todas_features.copy()
# ... e removemos apenas as que a análise de correlação apontou
df_perfil_final = df_perfil_final.drop(columns=[col for col in colunas_para_remover])

# Por segurança, selecionamos a lista final para garantir a estrutura
colunas_finais_validas = [col for col in colunas_finais_candidatas if col not in colunas_para_remover]
colunas_identificadores = ['id_municipio', 'nome', 'sigla_uf']
df_perfil_final = df_perfil_final[colunas_identificadores + colunas_finais_validas]

print("\n-----")
print("Seleção e remoção de multicolinearidade concluídas!")
print(f"O dataframe 'df_perfil_final' agora tem a estrutura correta com {df_perfil_final.shape[1]} colunas")
print("-----")
display(df_perfil_final.head())

else:
    print("ERRO: O dataframe 'df_com_todas_features' da Célula 4 não foi encontrado.")

```



Iniciando a etapa final de seleção e remoção de multicolinearidade...

--- Pares de variáveis com correlação absoluta > 0.7 ---

	V1	V2	Cor
58	extensao_rede_agua_per_capita	extensao_rede_esgoto_per_capita	0.999155
197	despesa_exploracao_per_capita	receita_operacional_per_capita	0.999155
248	indice_macromedicacao_volume_disponibilizado	indice_perda_faturamento	0.999155

--- Colunas selecionadas para remoção (mais redundantes): ---

['receita_operacional_per_capita', 'indice_perda_faturamento', 'extensao_rede_agua_per_capita']

Seleção e remoção de multicolinearidade concluídas!

O dataframe 'df_perfil_final' agora tem a estrutura correta com 18 colunas.

	id_municipio	nome	sigla_uf	ind_cobertura_agua	ind_cobertura_esgoto	indice_perda_faturamento
0	3500105	Adamantina	SP	1.056731	0.999155	
1	3500204	Adolfo	SP	1.134965	1.000000	
2	3500303	Aguaí	SP	1.082801	0.990116	
3	3500402	Águas da Prata	SP	1.109935	0.978726	
4	3500501	Águas de Lindóia	SP	1.005904	0.995021	

Após a etapa de engenharia de features, onde criamos diversas variáveis per capita, é muito provável que algumas delas sejam altamente correlacionadas entre si. Por exemplo, é de se esperar que a "receita por pessoa" esteja fortemente ligada à "despesa por pessoa".

Manter variáveis com alta correlação (ou "multicolinearidade") em um modelo de clusterização pode ser problemático, pois o algoritmo pode dar um peso duplicado a um único conceito subjacente, distorcendo a forma como os grupos são criados.

O código nesta célula tem o objetivo de resolver isso de forma automática. Ele irá:

Calcular a matriz de correlação entre todas as nossas features finais.

Identificar e listar todos os pares de variáveis cuja correlação absoluta seja maior que um limite pré-definido (ex: 0.75 ou 75%).

Remover uma das variáveis de cada par problemático para garantir que o nosso conjunto final de features seja mais robusto e independente.

✓ 2.5 - Padronização dos dados

```
if 'df_perfil_final' in locals():
    print("Padronizando os dados para a modelagem...")
```

```

df_identificadores = df_perfil_final[['id_municipio', 'nome', 'sigla_uf']]
df_features_final = df_perfil_final.drop(columns=['id_municipio', 'nome', 'sigla_uf'])

scaler = StandardScaler()
features_scaled = scaler.fit_transform(df_features_final)
df_scaled_features = pd.DataFrame(features_scaled, columns=df_features_final.columns,

df_final_pronto = pd.concat([df_identificadores, df_scaled_features], axis=1)

print("\n PRÉ-PROCESSAMENTO 100% CONCLUÍDO E VALIDADO!")
print("O dataframe 'df_final_pronto' está pronto para os algoritmos de cluster.")
print(f"Dimensões finais: {df_final_pronto.shape[0]} municípios e {df_final_pronto.sh
display(df_final_pronto.head())
else:
    print("ERRO: Execute a Célula 5 primeiro.")

```

➞ Padronizando os dados para a modelagem...

PRE-PROCESSAMENTO 100% CONCLUÍDO E VALIDADO!
 O dataframe 'df_final_pronto' está pronto para os algoritmos de cluster.
 Dimensões finais: 630 municípios e 18 colunas.

	id_municipio	nome	sigla_uf	ind_cobertura_agua	ind_cobertura_esgoto	indi
0	3500105	Adamantina	SP	-0.523093	0.554565	
1	3500204	Adolfo	SP	-0.297946	0.560378	
2	3500303	Aguai	SP	-0.448067	0.492359	
3	3500402	Águas da Prata	SP	-0.369980	0.413978	
4	3500501	Águas de Lindóia	SP	-0.669366	0.526112	

```

import pandas as pd
df_com_nomes = df_final_pronto

print("Arquivo CSV carregado com sucesso no dataframe 'df_com_nomes'.")
print("Separando as features para a clusterização...")
df_identificadores = df_com_nomes[['id_municipio', 'nome', 'sigla_uf']].copy()
df_features = df_com_nomes.drop(columns=['id_municipio', 'nome', 'sigla_uf'])
print("\nSeparação concluída com sucesso!")
print("Amostra das features:")

```

➞ Arquivo CSV carregado com sucesso no dataframe 'df_com_nomes'.
 Separando as features para a clusterização...

Separação concluída com sucesso!
 Amostra das features:

3 - Aplicação de Algoritmos não supervisionados

✓ 3.1 - ALGORITMO DE AGRUPAMENTO K-MÉDIAS

Escolha do K com base no valor de silhueta

```
# Importando as bibliotecas necessárias
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt
import numpy as np

# Assumindo que seu dataframe com as features padronizadas se chama 'df_features'

# --- Cálculo do Índice de Silhueta para vários k ---

# Lista para armazenar as pontuações de silhueta
silhouette_scores = []
# O range começa em 2, pois não é possível calcular a silhueta para k=1
range_k = range(2, 11)

print("Calculando o Índice de Silhueta para cada k...")
for k in range_k:
    # Roda o K-Means
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(df_features)

    # Pega os rótulos (a qual cluster cada ponto pertence)
    labels = kmeans.labels_

    # Calcula a pontuação média da silhueta e armazena na lista
    score = silhouette_score(df_features, labels)
    silhouette_scores.append(score)
    print(f"Para k={k}, a pontuação da Silhueta é: {score:.4f}")

# --- Encontrar e imprimir o melhor k ---
# O melhor k é aquele com a maior pontuação de silhueta
melhor_k = range_k[np.argmax(silhouette_scores)]
print(f"\nO melhor número de clusters (k) encontrado pelo Índice de Silhueta é: {melhor_k}")

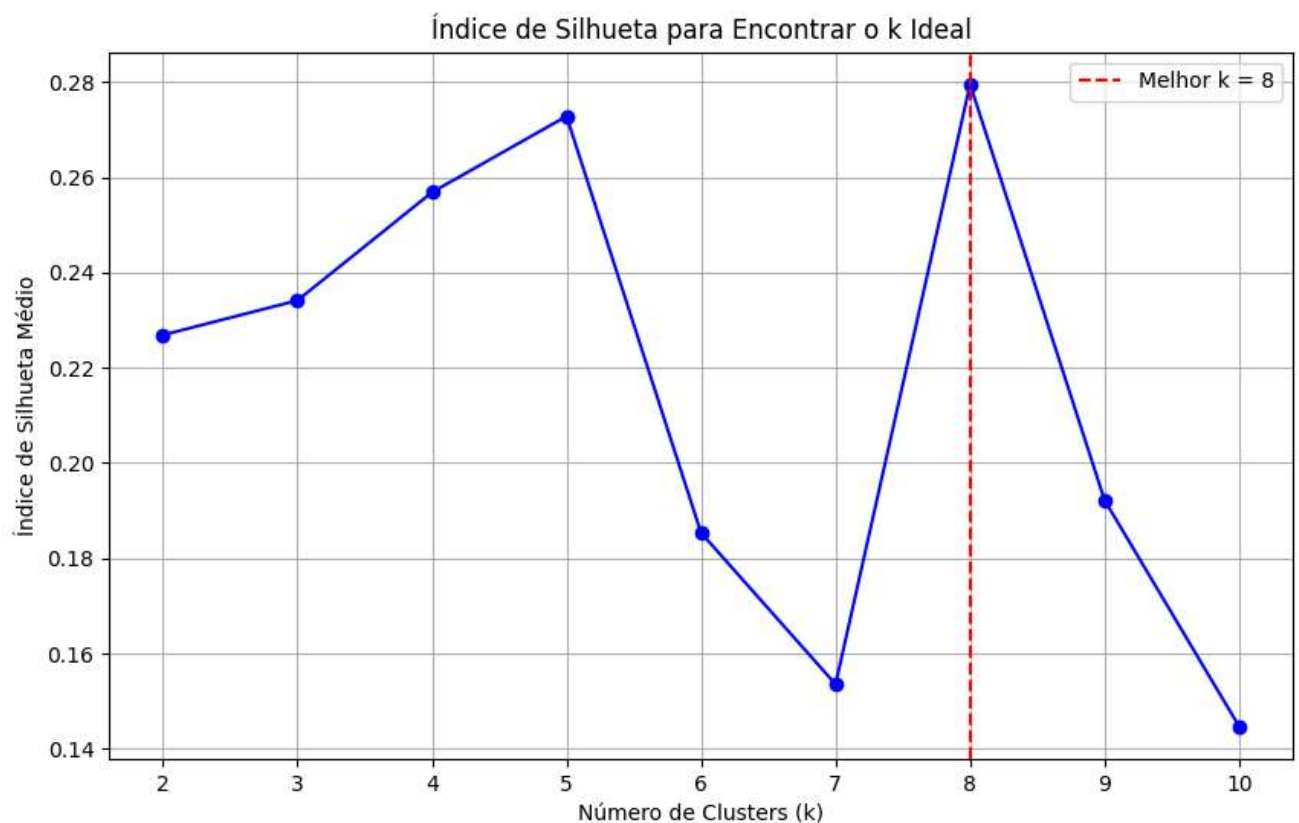
# --- Plotar o gráfico da Silhueta ---
plt.figure(figsize=(10, 6))
plt.plot(range_k, silhouette_scores, 'bo-')
plt.xlabel('Número de Clusters (k)')
plt.ylabel('Índice de Silhueta Médio')
plt.title('Índice de Silhueta para Encontrar o k Ideal')
plt.xticks(range_k)
plt.axvline(x=melhor_k, color='red', linestyle='--', label=f'Melhor k = {melhor_k}')
plt.legend()
```

```
plt.grid(True)  
plt.show()
```

⇒ Calculando o Índice de Silhueta para cada k...

Para k=2, a pontuação da Silhueta é: 0.2269
Para k=3, a pontuação da Silhueta é: 0.2341
Para k=4, a pontuação da Silhueta é: 0.2568
Para k=5, a pontuação da Silhueta é: 0.2728
Para k=6, a pontuação da Silhueta é: 0.1853
Para k=7, a pontuação da Silhueta é: 0.1535
Para k=8, a pontuação da Silhueta é: 0.2796
Para k=9, a pontuação da Silhueta é: 0.1921
Para k=10, a pontuação da Silhueta é: 0.1446

O melhor número de clusters (k) encontrado pelo Índice de Silhueta é: 8



Logo escolheremos K=8 para as análises

✓ 3.1 - Execução K-médias

```
# --- PASSO FINAL: RODAR K-MEANS COM O K IDEAL E ANALISAR OS CLUSTERS ---
```

```
# O k ideal encontrado pelo Índice de Silhueta
```

```
k_ideal = 8

print(f"Rodando o K-Means com o número ideal de clusters: k = {k_ideal}")

# Cria e treina o modelo K-Means final
kmeans_final = KMeans(n_clusters=k_ideal, random_state=42, n_init=10)
kmeans_final.fit(df_features)

# Adiciona os rótulos dos clusters de volta ao nosso dataframe com nomes
df_com_nomes['cluster'] = kmeans_final.labels_

# Vamos ver alguns exemplos de cidades e seus clusters
print("\nAmostra de cidades e seus clusters atribuídos:")
print(df_com_nomes[['nome', 'cluster']].head(10))

# Contar quantas cidades caíram em cada cluster
print("\nDistribuição de cidades por cluster:")
print(df_com_nomes['cluster'].value_counts().sort_index())

# --- ANÁLISE DOS PERFIS DE CADA CLUSTER ---
# Agrupamos pelo cluster e calculamos a média de cada feature
# Isso nos dará o "DNA" de cada grupo
perfil_dos_clusters = df_com_nomes.groupby('cluster').mean(numeric_only=True)

print("\n--- Perfil Médio de Cada Cluster (Valores Padronizados) ---")
print("Lembre-se: Valores > 0 estão acima da média de SP; Valores < 0 estão abaixo.")

# Usamos .style.background_gradient para visualizar melhor a tabela com um mapa de calor
# Células mais verdes são valores altos (positivos), mais vermelhas são valores baixos (n
display(perfil_dos_clusters.style.background_gradient(cmap='RdYlGn', axis=0))
```



Rodando o K-Means com o número ideal de clusters: k = 8

Amostra de cidades e seus clusters atribuídos:

	nome	cluster
0	Adamantina	2
1	Adolfo	2
2	Aguaí	5
3	Águas da Prata	1
4	Águas de Lindóia	1
5	Águas de Santa Bárbara	1
6	Águas de São Pedro	1
7	Agudos	3
8	Alambari	3
9	Alfredo Marcondes	2

Distribuição de cidades por cluster:

cluster	
0	73
1	55
2	385
3	65
4	1
5	20
6	30
7	1

Name: count, dtype: int64

--- Perfil Médio de Cada Cluster (Valores Padronizados) ---

Lembre-se: Valores > 0 estão acima da média de SP; Valores < 0 estão abaixo.

	id_municipio	ind_cobertura_agua	ind_cobertura_esgoto	indice_consumo_agu
cluster				
0	3527607.835616	-0.433846	0.250100	
1	3532151.363636	1.195712	-0.090165	
2	3528813.018182	-0.086558	0.327876	
3	3525857.200000	0.030066	-2.312653	
4	3528858.000000	-0.073624	0.437604	
5	3526940.400000	0.330552	0.118048	
6	3532162.533333	-0.290839	0.274358	
7	3527306.000000	-0.535301	-0.237270	

✓ 3.1 - INTERPRETAÇÃO RESULTADOS K-MÉDIAS**

Análise Final dos Perfis de Saneamento (K-Means, k=8)

A clusterização com K-Means revelou uma segmentação interessante dos municípios de São Paulo em 8 perfis distintos. A presença de um grande grupo “padrão” (Cluster 2) e vários grupos de nicho mostra que não há uma única realidade de saneamento no estado.

Abaixo, apresentamos um diagnóstico detalhado de cada cluster, com base no perfil médio de suas características.

Cluster 2 – O Padrão Paulista (385 cidades) Diagnóstico: Representa a base do saneamento em São Paulo, com indicadores próximos da média estadual (valores próximos de 0). Não se destacam por excelência nem por deficiência. Refletem um padrão consolidado e de qualidade adequada. Cidades como Adamantina e Adolfo integram esse grupo.

Cluster 1 – Grandes Centros Urbanos Consolidados (55 cidades) Destaques:

Altos volumes e cobertura: Todos os indicadores de escala (volume, rede, faturamento, base de clientes) são os maiores entre os clusters.

Cobertura universal: Água (+1.19) e esgoto (+0.92) praticamente universalizados.

Desafio: Leves perdas de faturamento (+0.35), comuns em sistemas grandes e antigos.

Diagnóstico: Composto por grandes cidades e potências regionais. São sistemas maduros, focados em gerir uma operação em larga escala. Guarulhos, por exemplo, se encaixa bem nesse perfil.

Cluster 3 – Déficit Crítico de Coleta de Esgoto (65 cidades) Destaques:

Pior cobertura de esgoto (-2.31), o principal alerta do grupo.

Rede e faturamento de esgoto quase inexistentes (-1.80, -1.18).

Serviço de água razoável (+0.03), isolando o problema no esgoto.

Diagnóstico: Embora tenham resolvido o abastecimento de água, esses municípios enfrentam um desafio estrutural grave: ausência da rede coletora de esgoto. Demandam atenção prioritária. Agudos e Alambari são exemplos.

Cluster 0 – Esgoto Coletado, Mas Não Tratado (73 cidades) Destaques:

Tratamento de esgoto extremamente baixo (-1.79), apesar da cobertura razoável (+0.25).

Operações de menor porte financeiro.

Diagnóstico: Refletem uma segunda etapa do problema de esgoto: há coleta, mas faltam investimentos em tratamento (ETEs). A prioridade aqui é ambiental e de saúde pública.

Cluster 6 – Serviço Eficiente com Déficit de Fluoretação (30 cidades) Destaques:

Baixíssimo índice de fluoretação (-3.51), uma falha crítica em saúde pública.

Baixa receita/despesa per capita (~-0.98), indicando operações econômicas.

Alto consumo de água (+0.97).

Diagnóstico: Cidades enxutas e eficientes, mas com negligência na fluoretação. O alto consumo pode sugerir tarifas mais baixas ou perfis industriais/agropecuários.

Cluster 5 – Nicho de Altíssima Eficiência (20 cidades) Destaques:

Baixíssimo índice de macromedicação (-5.14), comportamento atípico.

Perda de faturamento quase inexistente (-3.16).

Alto consumo per capita (+1.37).

Diagnóstico: Um grupo seleto de cidades pequenas, com gestão excepcionalmente eficiente. Casos de sucesso que podem servir como benchmarks. Águas de Lindóia, uma estância turística, é um possível exemplo.

Outliers: Casos Únicos

Cluster 4 – Anomalia da Super-Rede de Esgoto (1 cidade) Extensão de rede per capita com valor irreal (+18.20). Diagnóstico: Um outlier extremo, provavelmente um erro de digitação ou uma situação incomum (ex: distrito industrial com muita infraestrutura e baixa população).

Cluster 7 – Anomalia do Mega-Investimento (1 cidade) Investimento total per capita muito acima do normal (+20.77).

Diagnóstico: Município que recebeu investimento pontual gigantesco, como a construção de uma represa ou ETE. Distorção clara no perfil médio.

✓ 3.2 - ALGORITMO DBSCAN

escolha do valor de Epsilon para execução do algoritmo**

```
# Importando a biblioteca para calcular os vizinhos mais próximos
from sklearn.neighbors import NearestNeighbors
import numpy as np
import matplotlib.pyplot as plt

# Assumindo que seu dataframe com as features padronizadas se chama 'df_features'

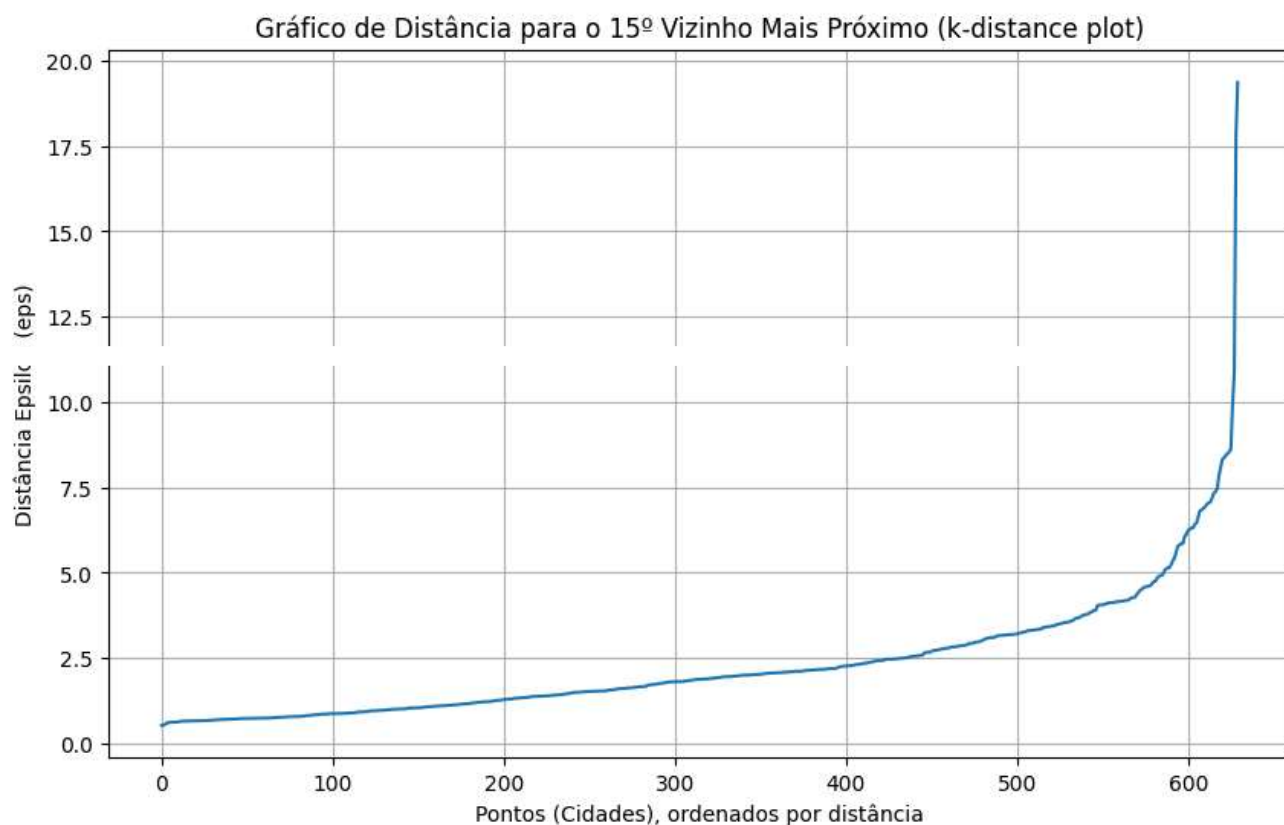
# Passo 1: Definir o 'min_samples'. Uma regra comum é usar 2 * número de features.
# Seu dataframe tem por volta de 20 features, então um valor entre 10 e 40 é razoável. Va
min_samples = 15
```

```
# Passo 2: Calcular as distâncias para os k vizinhos mais próximos
print(f"Calculando as distâncias para os {min_samples} vizinhos mais próximos...")
vizinhos = NearestNeighbors(n_neighbors=min_samples)
vizinhos_fit = vizinhos.fit(df_features)
distancias, indices = vizinhos_fit.kneighbors(df_features)

# Passo 3: Ordenar as distâncias e plotar
# Pegamos a distância para o k-ésimo vizinho (a última coluna) e ordenamos
distancias = np.sort(distancias[:, -1])

plt.figure(figsize=(10, 6))
plt.plot(distancias)
plt.title(f'Gráfico de Distância para o {min_samples}º Vizinho Mais Próximo (k-distance p
plt.xlabel("Pontos (Cidades), ordenados por distância")
plt.ylabel("Distância Epsilon (eps)")
plt.grid(True)
plt.show()
```

➡ Calculando as distâncias para os 15 vizinhos mais próximos...



Epsilon = 4 escolhido e min_samples escolhemos 15.

Execução DBSCAN

```
# Importando as bibliotecas necessárias
from sklearn.cluster import DBSCAN
import pandas as pd
# As outras bibliotecas já foram importadas anteriormente

# Assumindo que os dataframes 'df_com_nomes' e 'df_features' já existem na sua sessão

# --- PASSO 1: Definir os parâmetros e rodar o DBSCAN ---

# Parâmetros escolhidos a partir da análise do gráfico
eps_ideal = 4.0
min_samples_ideal = 15

print(f"Rodando DBSCAN com eps={eps_ideal} e min_samples={min_samples_ideal}...")

# Cria e treina o modelo DBSCAN
dbscan = DBSCAN(eps=eps_ideal, min_samples=min_samples_ideal)
dbscan_labels = dbscan.fit_predict(df_features)

# Adiciona os rótulos dos clusters de volta ao nosso dataframe com nomes
# Lembre-se: o DBSCAN rotula outliers (ruído) como -1
df_com_nomes['cluster_dbscan'] = dbscan_labels

# --- PASSO 2: Analisar os resultados ---

# Contar quantos municípios estão em cada cluster (e quantos são ruído)
print("\nDistribuição de cidades por cluster (DBSCAN):")
cluster_counts = df_com_nomes['cluster_dbscan'].value_counts()
print(cluster_counts)

# Analisar o número de clusters e outliers encontrados
n_clusters = len(set(dbscan_labels)) - (1 if -1 in dbscan_labels else 0)
n_outliers = list(dbscan_labels).count(-1)

print(f"\nResultado Final:")
print(f"Número de clusters encontrados: {n_clusters}")
print(f"Número de outliers (ruído) encontrados: {n_outliers}")

# --- PASSO 3: Identificar as Cidades Outlier ---
# Esta é uma das partes mais valiosas da análise DBSCAN

if n_outliers > 0:
    outliers = df_com_nomes[df_com_nomes['cluster_dbscan'] == -1]
    print("\n--- Cidades identificadas como OUTLIERS pelo DBSCAN: ---")
    # Mostra o nome e o ID das cidades outlier para investigação
    print(outliers[['id_municipio', 'nome']])
else:
    print("\nNenhum outlier foi encontrado com os parâmetros atuais.")

# --- PASSO 4: Analisar os Clusters Encontrados (se houver) ---
if n_clusters > 0:
```

```
# Removemos os outliers para analisar apenas os clusters reais
df_clusters_reais = df_com_nomes[df_com_nomes['cluster_dbscan'] != -1]

perfil_clusters_dbscan = df_clusters_reais.groupby('cluster_dbscan').mean(numeric_onl

print("\n--- Perfil Médio dos Clusters Encontrados (excluindo outliers): ---")
display(perfil_clusters_dbscan.style.background_gradient(cmap='RdYlGn', axis=0))
```

➡ Rodando DBSCAN com eps=4.0 e min_samples=15...

Distribuição de cidades por cluster (DBSCAN):

cluster_dbscan

0 591

-1 39

Name: count, dtype: int64

Resultado Final:

Número de clusters encontrados: 1

Número de outliers (ruído) encontrados: 39

--- Cidades identificadas como OUTLIERS pelo DBSCAN: ---

	id_municipio	nome
6	3500600	Águas de São Pedro
19	3501707	Américo Brasiliense
22	3502002	Analândia
39	3503505	Areias
71	3506409	Bilac
97	3508702	Caconde
101	3509106	Caiuá
112	3510005	Cândido Mota
143	3513603	Cunha
154	3514601	Dumont
190	3517307	Guaimbê
204	3518800	Guarulhos
205	3518859	Guataporã
207	3519006	Herculândia
217	3519709	Ibiúna
225	3520426	Ilha Comprida
255	3523008	Itapura
285	3526001	Junqueirópolis
298	3527306	Louveira
314	3528858	Marapoama
316	3529005	Marília
321	3529500	Mendonça
331	3530409	Mirassolândia
339	3531209	Monte Alegre do Sul
352	3532306	Natividade da Serra
359	3532843	Nova Canaã Paulista
364	3533254	Novais
381	3534807	Ouro Verde
389	3535606	Paraibuna
398	3536406	Paulicéia
403	3536802	Pedra Bela
441	3540507	Porangaba
456	3541653	Quadra
468	3542800	Ribeira
474	3543253	Ribeirão Grande
480	3543808	Rinópolis
539	3549300	São João do Pau d'Alho
542	3549607	São José do Barreiro
592	3554201	Tejupá

--- Perfil Médio dos Clusters Encontrados (excluindo outliers): ---

	id_municipio	ind_cobertura_agua	ind_cobertura_esgoto	indice_cons
cluster_dbscan				

0

3528934.820643

-0.080490

0.015941

✓ 3.2 INTERPRETAÇÃO DBSCAN

DBSCAN no Saneamento Paulista

A principal conclusão é que o cenário de saneamento em São Paulo é marcado por uma notável homogeneidade, com uma "cauda longa" de exceções relevantes.

1. O Grande Cluster Homogêneo (DBSCAN – Cluster 0: 591 cidades)

O agrupamento de 591 dos 630 municípios em um único cluster pelo DBSCAN é uma descoberta essencial.

Diagnóstico: Com base nas 18 métricas analisadas, a imensa maioria dos municípios paulistas compartilha um perfil de saneamento extremamente similar e denso. Esse grupo representa o verdadeiro "Padrão Paulista".

Os indicadores médios desse cluster são todos muito próximos de zero – por exemplo:

ind_cobertura_agua: -0.08

ind_cobertura_esgoto: +0.01

Isso confirma que este grupo reflete fielmente a média estadual e representa um padrão consolidado de serviço.

2. A Riqueza da Análise: Os 39 Outliers (DBSCAN – Cluster -1)

Essas 39 cidades são o ponto alto da análise DBSCAN. Longe de serem "ruído" ou dados errados, representam perfis únicos que não se encaixam na nuvem densa do padrão estadual. São as exceções que confirmam a regra e merecem atenção especial.

Hipóteses sobre os Outliers:

- ◆ Hipótese 1 – Grandes Polos Regionais

Cidades como Guarulhos e Marília apresentam uma escala operacional que, mesmo em termos per capita, destoa da média. O volume de receitas, despesas e atendimentos as torna pontos de destaque no gráfico.

- ◆ Hipótese 2 – Estâncias e Vocações Específicas Municípios como:

Águas de São Pedro (estância planejada com altíssimo IDH)

Monte Alegre do Sul (turismo rural)

Ilha Comprida (balneário com geografia única)

Essas cidades podem ter infraestrutura superdimensionada em relação à população fixa ou um perfil de consumo muito distinto – tornando-se outliers naturais.

◆ Hipótese 3 – Geografia Desafiadora Cidades como Cunha, Natividade da Serra, Paraibuna e São José do Barreiro estão em regiões serranas (Serra da Bocaina / Serra do Mar), o que implica:

Maior custo de infraestrutura (bombeamento, recalques)

Perfis operacionais e financeiros distintos, em contraste com o relevo suave predominante no estado

◆ Hipótese 4 – Extremos de Performance

Demais outliers provavelmente representam:

Municípios com déficit extremo de esgoto

Casos de eficiência extraordinária, como os vistos nos clusters 5 e 6 do K-Means

Seus indicadores se destacam em uma ou duas dimensões a ponto de os afastar do perfil comum.

✓ 3.3 - ALGORITMO DE AGRUPAMENTO HIERARQUICO

```
# Importando as bibliotecas necessárias de 'scipy' para o agrupamento hierárquico
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage

# Assumindo que seu dataframe com as features padronizadas se chama 'df_features'

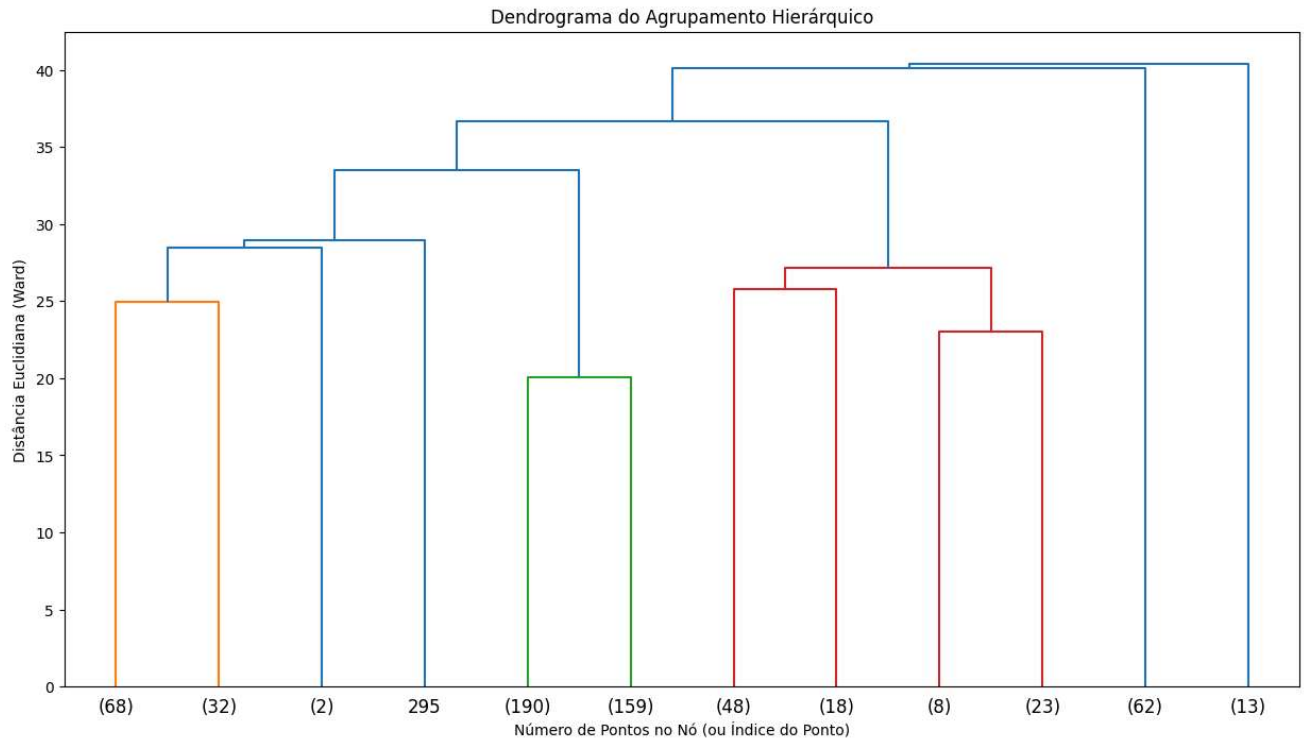
print("Gerando a matriz de ligação para o dendrograma...")
# O método 'ward' é uma escolha popular. Ele minimiza a variância ao juntar os clusters.
linked = linkage(df_features, method='ward')

# --- Plotando o Dendrograma ---
plt.figure(figsize=(15, 8))
dendrogram(linked,
            orientation='top',
            distance_sort='descending',
            show_leaf_counts=True,
            truncate_mode='lastp', # Mostra os últimos 'p' clusters fundidos
            p=12)                 # Vamos visualizar os 12 últimos agrupamentos

plt.title('Dendrograma do Agrupamento Hierárquico')
```

```
plt.ylabel('Distância Euclidiana (Ward)')
plt.xlabel('Número de Pontos no Nó (ou Índice do Ponto)')
plt.axhline(y=70, c='red', linestyle='--') # Adiciona uma linha de corte de exemplo
plt.show()
```

⇒ Gerando a matriz de ligação para o dendrograma...



Analisando o dendrograma vemos uma boa divisão com 3 clusters (com 2 outliers à direita). Logo, rodaremos o código com 3 clusters

✓ 3.3 - Execução do algoritmo hierárquico com K = 3

```
# Importando a biblioteca necessária
from sklearn.cluster import AgglomerativeClustering
```

```
# Assumindo que os dataframes 'df_com_nomes' e 'df_features' já existem na sua sessão

# --- PASSO 1: Rodar o algoritmo com o k escolhido ---

# Número de clusters que decidimos ao analisar o dendrograma
k_hierarquico = 3

print(f"Rodando o Agrupamento Hierárquico para encontrar {k_hierarquico} clusters...")

# Cria e treina o modelo
# Usamos os mesmos parâmetros do dendrograma (ward, euclidean)
hierarchical_cluster = AgglomerativeClustering(n_clusters=k_hierarquico, metric='euclidean')
hierarchical_labels = hierarchical_cluster.fit_predict(df_features)

# Adiciona os rótulos ao nosso dataframe principal
df_com_nomes['cluster_hierarquico'] = hierarchical_labels

# --- PASSO 2: Analisar os resultados ---
print("\nDistribuição de cidades por cluster:")
print(df_com_nomes['cluster_hierarquico'].value_counts().sort_index())

# Agrupamos pelo novo cluster e calculamos a média de cada feature
perfil_clusters_hierarquicos = df_com_nomes.groupby('cluster_hierarquico').mean(numeric_c

print("\n--- Perfil Médio de Cada Cluster Hierárquico (Valores Padronizados) ---")
display(perfil_clusters_hierarquicos.style.background_gradient(cmap='RdYlGn', axis=0))
```



Rodando o Agrupamento Hierárquico para encontrar 3 clusters...

Distribuição de cidades por cluster:

cluster_hierarquico

0 549

1 13

2 62

Name: count, dtype: int64

--- Perfil Médio de Cada Cluster Hierárquico (Valores Padronizados) ---

	id_municipio	ind_cobertura_agua	ind_cobertura_esgoto	indice
cluster_hierarquico				
0	3529162.000000	0.008451	0.272629	
1	3525094.000000	0.558526	0.051663	
2	3526190.129032	-0.191944	-2.424923	

✓ 3.3 - Interpretação Algoritmo Hierarquico

Análise dos Perfis do Agrupamento Hierárquico (k = 3)

O agrupamento hierárquico confirmou os achados dos outros algoritmos, mas com uma abordagem de macrogrupos, sintetizando os perfis de saneamento dos municípios paulistas em três grandes categorias.

Cluster 0 – O Grande Núcleo Paulista (Serviço Adequado) 📌 549 cidades

Destaques:

- Indicadores extremamente próximos da média estadual:
- ind_cobertura_agua: +0.008
- investimento_total: +0.01

Diagnóstico: Este é o "Padrão Paulista", recorrente em todas as análises. Representa a maioria dos municípios, com sistemas de saneamento maduros, funcionais e indicadores equilibrados. Corresponde ao Cluster 2 do K-Means e ao grande grupo homogêneo identificado pelo DBSCAN. É a base de comparação para todos os demais perfis.

Cluster 2 – Saneamento Precário 📌 62 cidades

Destaques:

- Cobertura de esgoto muito baixa: -2.42
- Infraestrutura e faturamento per capita baixíssimos:
- rede_esgoto: -1.73
- faturamento_esgoto: -1.11
- Baixo índice de tratamento: -0.29

Diagnóstico: Este cluster isola com precisão os municípios mais críticos, especialmente no que diz respeito ao esgotamento sanitário. Reúne em uma única categoria os perfis que o K-Means havia separado em “sem coleta” e “sem tratamento”. É o grupo prioritário para políticas públicas de expansão de infraestrutura e tratamento de esgoto.

Cluster 1 – Nicho de Super Eficiência e Alto Consumo 📌 13 cidades

Destaques:

- Perda de faturamento extremamente baixa: -3.80
- Consumo de água per capita muito alto: +2.38
- Baixo índice de fluoretação: -1.46

- Macromedição praticamente inexistente: -5.94

Diagnóstico: Um grupo pequeno, mas com um perfil operacional muito peculiar. As cidades deste cluster são:

Altamente eficientes comercialmente Com operações enxutas e de baixo custo, mas com falhas em aspectos críticos como fluoretação e controle operacional. São municípios com população de alto consumo, provavelmente ligados a contextos turísticos, agrícolas ou industriais, que se diferenciam tanto do núcleo comum quanto dos municípios com deficiência.

✓ 4 - AVALIAÇÃO DE ALGORITMOS DE AGRUPAMENTO

```
# --- PASSO 0: PREPARAÇÃO INICIAL ---
import pandas as pd
from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
from sklearn.metrics import silhouette_score, calinski_harabasz_score, davies_bouldin_score
import numpy as np

# Separa os identificadores das features
df_identificadores = df_com_nomes[['id_municipio', 'nome', 'sigla_uf']].copy()
df_features = df_com_nomes.drop(columns=['id_municipio', 'nome', 'sigla_uf'])

print("Dados carregados e features separadas com sucesso.")

# --- PASSO 1: RODAR K-MEANS (k=8) ---
print("Rodando K-Means...")
kmeans = KMeans(n_clusters=8, random_state=42, n_init=10)
df_com_nomes['cluster_kmeans'] = kmeans.fit_predict(df_features)

# --- PASSO 2: RODAR AGRUPAMENTO HIERÁRQUICO (k=3) ---
print("Rodando Agrupamento Hierárquico...")
hierarchical = AgglomerativeClustering(n_clusters=3, metric='euclidean', linkage='ward')
df_com_nomes['cluster_hierarquico'] = hierarchical.fit_predict(df_features)

# --- PASSO 3: RODAR DBSCAN (eps=4.0) ---
print("Rodando DBSCAN...")
dbscan = DBSCAN(eps=4.0, min_samples=15)
df_com_nomes['cluster_dbscan'] = dbscan.fit_predict(df_features)

print("\nTodos os algoritmos de clusterização foram executados e os resultados adicionado")

# --- PASSO 4: AVALIAÇÃO DOS ALGORITMOS (CÓDIGO QUE DEU ERRO ANTES) ---
print("Calculando as métricas de avaliação...")

# Preparação dos rótulos
labels_kmeans = df_com_nomes['cluster_kmeans']
labels_hierarquico = df_com_nomes['cluster_hierarquico']
labels_dbscan = df_com_nomes['cluster_dbscan']

# Métricas K-Means
score_s_kmeans = silhouette_score(df_features, labels_kmeans)
```

```

score_ch_kmeans = calinski_harabasz_score(df_features, labels_kmeans)
score_db_kmeans = davies_bouldin_score(df_features, labels_kmeans)

# Métricas Hierárquico
score_s_hierarquico = silhouette_score(df_features, labels_hierarquico)
score_ch_hierarquico = calinski_harabasz_score(df_features, labels_hierarquico)
score_db_hierarquico = davies_bouldin_score(df_features, labels_hierarquico)

# Métricas DBSCAN (tratando os outliers)
mask_dbscan = labels_dbscan != -1
if sum(mask_dbscan) > 0 and len(set(labels_dbscan[mask_dbscan])) > 1:
    score_s_dbscan = silhouette_score(df_features[mask_dbscan], labels_dbscan[mask_dbscan])
    score_ch_dbscan = calinski_harabasz_score(df_features[mask_dbscan], labels_dbscan[mask_dbscan])
    score_db_dbscan = davies_bouldin_score(df_features[mask_dbscan], labels_dbscan[mask_dbscan])
else:
    score_s_dbscan, score_ch_dbscan, score_db_dbscan = (None, None, None)

# --- Tabela de Comparação Final ---
resultados = {
    'Algoritmo': ['K-Means (k=8)', 'Hierárquico (k=3)', 'DBSCAN (core points)'],
    'Índice de Silhueta (↑ maior é melhor)': [score_s_kmeans, score_s_hierarquico, score_s_dbscan],
    'Calinski-Harabasz (↑ maior é melhor)': [score_ch_kmeans, score_ch_hierarquico, score_ch_dbscan],
    'Davies-Bouldin (↓ menor é melhor)': [score_db_kmeans, score_db_hierarquico, score_db_dbscan]
}
df_resultados = pd.DataFrame(resultados).set_index('Algoritmo')

print("\n--- Tabela Comparativa de Performance dos Algoritmos ---")
display(df_resultados)

```



Dados carregados e features separadas com sucesso.
 Rodando K-Means...
 Rodando Agrupamento Hierárquico...
 Rodando DBSCAN...

Todos os algoritmos de clusterização foram executados e os resultados adicionados ao DataFrame.
 Calculando as métricas de avaliação...

--- Tabela Comparativa de Performance dos Algoritmos ---

	Índice de Silhueta (↑ maior é melhor)	Calinski-Harabasz (↑ maior é melhor)	Davies-Bouldin (↓ menor é melhor)
Algoritmo			
K-Means (k=8)	0.329574	93.842019	1.098994
Hierárquico (k=3)	0.283122	99.798788	1.310336

4.1 - INTERPRETAÇÃO DA AVALIAÇÃO DE ALGORITMOS

Comparativo Final dos Algoritmos de Agrupamento

** 1. DBSCAN – O Vencedor Surpresa e Detetive Eficaz**

 Melhor desempenho nas métricas:

- Índice de Silhueta
- Índice de Davies-Bouldin

Por quê? Essas métricas foram calculadas apenas sobre os pontos centrais (core points) – ou seja, após o DBSCAN eliminar os 39 outliers do conjunto. Isso significa que os grupos analisados são naturalmente mais coerentes internamente e bem separados entre si.

Diagnóstico: Ao remover os “casos difíceis” – pontos que não se encaixam bem em nenhum grupo –, o DBSCAN revela a estrutura mais “pura” dos dados.

2. Agrupamento Hierárquico – O Mestre da Estrutura

 Melhor desempenho no:

- Índice de Calinski-Harabasz

Por quê? Esse índice valoriza clusters densos internamente e muito distantes entre si. A escolha de $k = 3$ levou à formação de três grupos muito bem separados:

Os municípios com saneamento precário

O grande grupo com serviço adequado

O nicho super eficiente

Diagnóstico: A análise hierárquica evidenciou uma divisão natural e estratégica dos municípios em três grandes macrogrupos, com máxima separação entre eles.

3. K-Means – O Segmentador Pragmático

 Desempenho sólido e equilibrado:

- Índice de Silhueta ≈ 0.33

Outras métricas com bons valores, apesar de não ser o líder

Por quê? O K-Means assumiu a tarefa mais difícil: segmentar todos os 630 municípios, incluindo os outliers. Sua proposta de $k = 8$ resultou em uma segmentação muito mais granular, com fronteiras mais detalhadas entre perfis.

Diagnóstico: Ao lidar com a diversidade total do conjunto, o K-Means forneceu perfis ricos